# Lecture 3

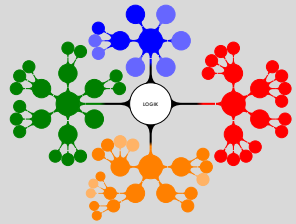
# Introduction to web application development with Python

## Web development, python frameworks, and basic web security measures

*Application Development with Python* of April 30, 2026

Evrad Kamtchoum
Centre for Cybersecurity and Mathematical Cryptology
The University of Bamenda

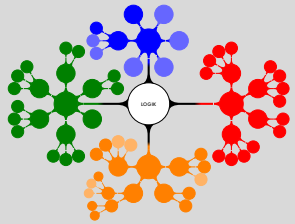




# Objectives of this lecture

1. Introduce the basics of web development and the role of Python in creating web applications
2. Discuss the various frameworks and librairies available for web development with python
3. Demonstrate how to setup a web development environment using Python and a web server
4. Learn how to create a simple web application using Python and a framework like Django/Flask
5. Address common challenges and best practices in web development with Python, such as security considerations

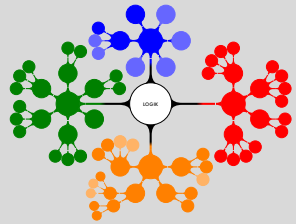




# Content







# How the Web Works: Client-Server Model

## Client-erver Architecture

Web applications follow a **client-server model**:

- **Client**: The user's device (browser) that sends requests
- **Server**: Processes requests and returns responses

## HTTP Protocol (HyperText Transfer Protocol)

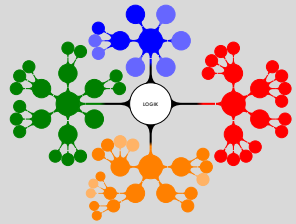
Communication between client and server is done using **HTTP**:

- **Request methods**: GET, POST, PUT, DELETE
- **Status codes**:
  - 200 (OK), 404 (Not Found), 500 (Server Error)

## Request-Response Lifecycle

1. User enters a URL in the browser
2. Browser sends an **HTTP request** to the server
3. Server processes the request (logic + database)
4. Server returns an **HTTP response** (HTML/JSON)
5. Browser renders the result





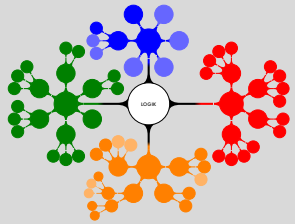
# Web development

Web development refers to the process of creating and building websites and web applications that are accessible over the Internet. It involves the use of various technologies, programming languages, and tools to design, develop, and maintain websites that cater to different purposes and functionalities.

**At its core, web development encompasses two main aspects:**

- **Front-end development**
- **Back-end development**



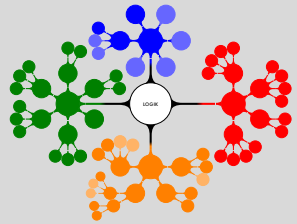


# Front-end, Back-end development

- Front-end development focuses on the visual and interactive elements of a website that users directly interact with. This includes designing the user interface (UI) and user experience (UX), creating the layout, and implementing the visual components using technologies such as HTML (Hypertext Markup Language), CSS (Cascading Style Sheets), and JavaScript. Front-end developers ensure that websites are visually appealing, responsive, and user-friendly

- Back-end development, on the other hand, deals with the server-side functionality of a website. It involves creating the logic and infrastructure that support the website's functionality, handle data processing, and interact with databases. Back-end developers work with programming languages such as Python, Ruby, Java, or PHP to build the server-side components of a web application.





# What is Python Web Development?

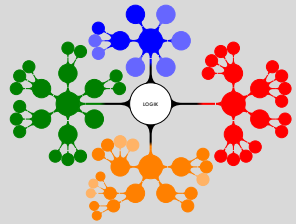
## Python web development

Python web development refers to the process of creating web applications and websites using the Python programming language. Python is a versatile and popular programming language known for its simplicity, readability, and vast ecosystem of libraries and frameworks.

## Python as a back-end development langauge

In Python web development, developers use Python to write the server-side logic that powers web applications. This includes handling HTTP requests and responses, managing data storage and retrieval, implementing business logic, and rendering dynamic content.





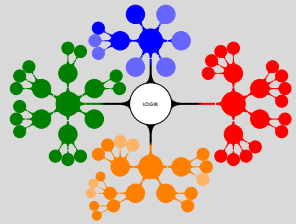
# Why use Python for Web Development?

Python is a popular programming language that has gained significant traction in web development. It offers several advantages, making it an excellent choice for building robust and scalable web applications. Here are some compelling reasons why Python is widely used for web development:

- **Readability and Simplicity**: Python's syntax is designed to be easy to read and write, emphasizing code readability and maintainability. Its clean and intuitive syntax allows developers to express concepts in fewer lines of code, making development faster and more efficient. Python's simplicity enables both beginners and experienced developers to work with ease and collaborate effectively

- **Large and Active Community**: Python has a vast and active community of developers who contribute to its growth and offer support. The community provides numerous libraries, frameworks, and resources specifically tailored for web development. This abundance of community-driven tools and resources makes Python a powerful choice for web development, offering solutions for a wide range of requirements.



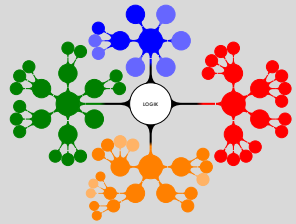


# Why use Python for Web Development? (2)

- **Extensive Libraries and Frameworks**: Python offers a rich ecosystem of libraries and frameworks that simplify web development tasks. Django, one of the most popular Python web frameworks, provides a complete and robust set of tools for building complex web applications. It follows the Model-View-Controller (MVC) architectural pattern and offers features like authentication, database ORM (Object-Relational Mapping), and URL routing out of the box. Flask is another lightweight and flexible micro-framework that allows developers to have more control over the application's structure and components. These frameworks, along with others like Pyramid and Bottle, provide a solid foundation and enhance productivity in web development.

- **Scalability and Performance**: Python is known for its scalability and performance, making it suitable for handling high-traffic web applications. With advancements like asynchronous programming, Python frameworks like **Django** and **asyncio** can efficiently handle concurrent requests and maximize server resources. Additionally, Python's integration capabilities allow easy integration with other languages, enabling developers to leverage high-performance libraries written in C or C++ when needed.





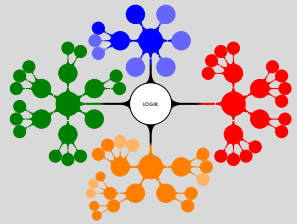
# Why use Python for Web Development? (3)

- **Integration and Compatibility**: Python seamlessly integrates with other technologies, making it flexible for web development. It supports various databases, including SQL-based databases like MySQL, PostgreSQL, and SQLite, as well as NoSQL databases like MongoDB. Python's compatibility extends to web servers, message queues, caching systems, and APIs, allowing developers to integrate different components and services smoothly.

- **Testing and Debugging**: Python offers robust testing frameworks, such as **unittest** and **pytest**, which simplify the process of writing and executing tests for web applications. Its debugging tools, like **pdb** and integrated development environments (IDEs), provide effective debugging capabilities, helping developers identify and fix issues quickly.

- **Rapid Development**: Python's focus on simplicity and productivity enables developers to build web applications quickly. The availability of pre-built modules and libraries allows developers to leverage existing solutions and avoid reinventing the wheel. This rapid development approach is particularly beneficial for startups and small-scale projects, where time-to-market is crucial.





# Architecture Patterns: MVC vs Django MTV

## Model-View-Controller (MVC)

A common architectural pattern in web development:

- **Model**: Manages data and business logic
- **View**: Displays data (UI)
- **Controller**: Handles user input and updates Model/View

## Django's Model-Template-View (MTV)

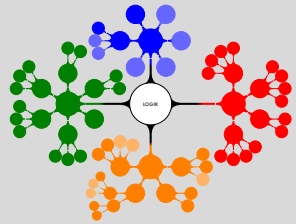
Django uses a slightly different pattern:

- **Model**: Manages data (same as MVC)
- **Template**: Handles presentation (HTML pages)
- **View**: Contains business logic and controls data flow

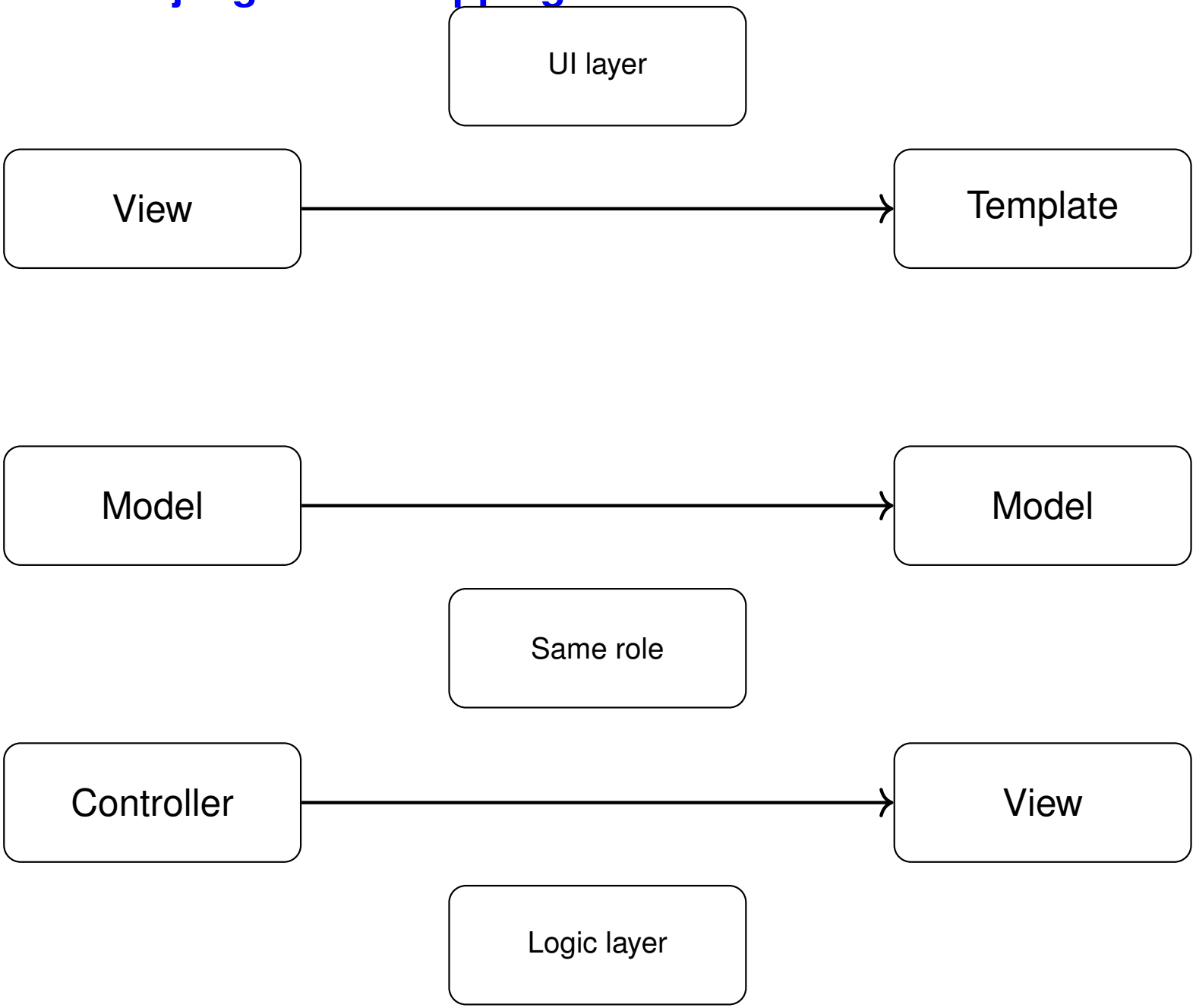
## Key Difference

- In **MVC**, the **Controller** handles logic
- In **Django**, the **View** plays the role of the Controller
- Templates replace traditional Views (UI layer)

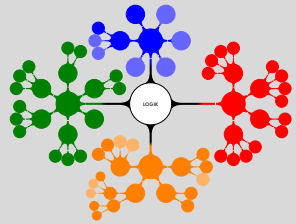




# MVC vs Django MTV Mapping





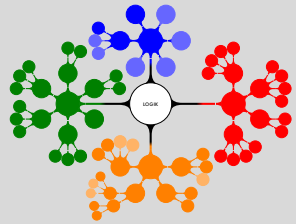
# How to use Python for Web Development

Python is a versatile programming language that can be used for python web application development. With its simplicity, readability, and vast ecosystem of libraries and frameworks, Python has become a popular choice for building web applications. Here are the key steps to get started with Python for web development:

## Steps

1. Install Python
2. Choose a Web Framework
3. Set Up a Development Environment
4. Install Framework and Dependencies
5. Project Initialization
6. Configure Settings
7. Define Models
8. Create Views and Templates
9. Define URL Routes
10. Handle Forms and User Input
11. Integrate with Databases
12. Implement Business Logic
13. Test and Debug
14. Deploy and Maintain





# Databases in Web Applications

## Why Databases?

Web applications need to store and manage data such as:

- Users (accounts, passwords)
- Products, orders, messages, etc.

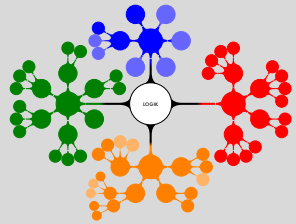
## Basic Concepts

- **Database**: Organized collection of data
- **Table**: Structure storing rows and columns
- **Record**: A single row in a table
- **Schema**: Structure of the database

## CRUD Operations

- **Create**, **Read**, **Update**, **Delete**





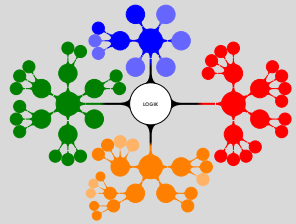
# Database Relationships

- **One-to-One**: One user ? one profile
- **One-to-Many**: One user ? many orders
- **Many-to-Many**: Students ? Courses

## ORM (Object-Relational Mapping)

- Interact with database using Python objects
- Example: Django ORM, SQLAlchemy





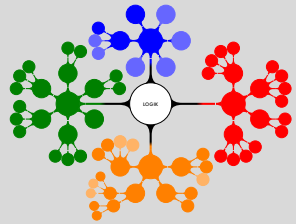
# Different Python Web Development Frameworks

Python provides a number of web frameworks to meet a variety of purposes and tastes. Here are some popular Python web development frameworks:

- **Django**: Django is a high-level, full-featured framework known for its "batteries included" approach.It offers a rich collection of tools and functionality for swiftly developing complicated web applications. Django includes an ORM (Object-Relational Mapping) for database management, URL routing, form handling, authentication, and more. It follows the Model-View-Controller (MVC) architectural pattern.

- **Flask**: Flask is a lightweight and flexible framework, often referred to as a micro-framework. It provides the essentials for python web application development, allowing developers to have more control over their application's structure. Flask supports URL routing, template rendering, and request handling, but leaves other functionalities like database management and authentication to extensions and libraries. It follows a minimalist approach and is ideal for small to medium-sized projects.



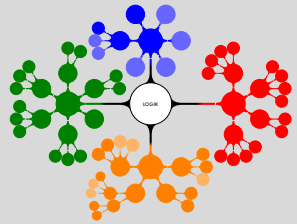


# Different Python Web Development Frameworks (2)

- **Pyramid**: Pyramid is a flexible and scalable web framework that aims to strike a balance between simplicity and power. It follows a minimalist philosophy and allows developers to choose the components they need. Pyramid supports various templating engines, and URL dispatching, and includes tools for authentication, caching, and internationalization. It is suitable for projects of any size, from small applications to large-scale enterprise systems.

- **Bottle**: Bottle is a minimalistic web framework with a small footprint. It is designed to be easy to learn and use, making it a good choice for beginners or small projects. Despite its simplicity, Bottle provides routing, template rendering, and basic tools for handling HTTP requests and responses. It is a single-file module with no external dependencies, making it easy to deploy and distribute.





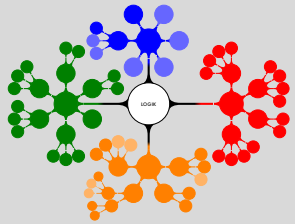
# Different Python Web Development Frameworks (3)

- **CherryPy**: CherryPy is a minimalist web framework that aims to be fast, stable, and scalable. It provides a simple and intuitive API for handling HTTP requests, URL routing, and session management. CherryPy can run as a standalone HTTP server or be integrated with other servers. It is suitable for building small to medium-sized applications and APIs.

- **Tornado**: Tornado is a powerful and scalable web framework with a focus on performance and handling high-traffic applications. It is designed for building asynchronous web servers and supports non-blocking I/O operations. Tornado can handle thousands of simultaneous connections efficiently and is suitable for applications that require real-time functionality, such as chat servers or streaming platforms.

These are just a few examples of Python web frameworks, and each has its own strengths and use cases. Consider your project requirements, scalability needs, learning curve, and community support when choosing the framework that best fits your development goals.





# Modern Python Framework: FastAPI

## What is FastAPI?

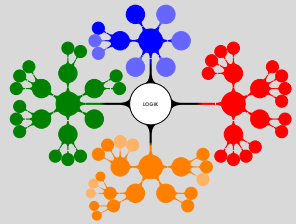
FastAPI is a modern, high-performance web framework for building APIs with Python.

- Built on **ASGI** (asynchronous support)
- Very fast (comparable to Node.js and Go)
- Automatic API documentation (Swagger UI)
- Uses Python type hints for validation

## Use Cases

- REST APIs
- Microservices
- High-performance applications



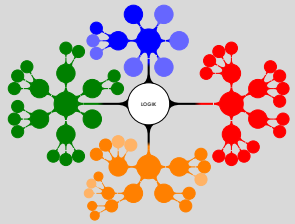


# Python libraries for Web Development

Web development using Python offers a wide range of tools and libraries that can enhance your productivity and simplify the development process. Here are some commonly used tools & libraries in Python web development:

- **Requests**: Requests is a simple and user-friendly library for making HTTP requests. It simplifies interacting with web APIs and handling HTTP methods, headers, cookies, and authentication.
- **Beautiful Soup**: Beautiful Soup is an HTML and XML parsing library. It provides a simple API for exploring and modifying the parsed data, making it suitable for web scraping and information extraction from web pages.
- **Pillow**: Pillow is a powerful library for image processing and manipulation. It provides functionalities like resizing, cropping, applying filters, and adding text or overlays to images. The pillow is often used in web applications for image handling and manipulation.
- **SQLAlchemy**: SQLAlchemy is a feature-rich ORM (Object-Relational Mapping) library that simplifies database management in Python. It supports multiple database engines and provides a high-level API for interacting with databases, making it easier to work with databases in web applications.
- **Celery**: Celery is a distributed task queue library that enables asynchronous task execution in web applications. It allows you to offload time-consuming or resource-intensive tasks to be processed in the background, improving the responsiveness of your application.
- **Flask-SQLAlchemy**: Flask-SQLAlchemy is an extension that integrates SQLAlchemy with the Flask web framework. It provides seamless integration between Flask and SQLAlchemy, making it easier to work with databases in Flask applications.

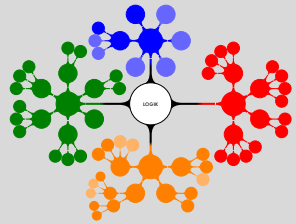




# Comparison of Python Web Frameworks

| Framework | Type | Use Case |
|---|---|---|
| Django | Full-stack | Large, complex applications |
| Flask | Micro-framework | Small to medium projects |
| FastAPI | Async framework | APIs and high-performance apps |
| Pyramid | Flexible | Scalable applications |
| Bottle | Minimalist | Simple applications |
| Tornado | Async | Real-time systems |

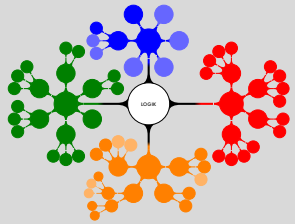




# Tools and Technologies Overview

| Category | Examples |
|---|---|
| Frameworks | Django, Flask, FastAPI |
| Databases | MySQL, PostgreSQL, SQLite |
| ORM | SQLAlchemy, Django ORM |
| Version Control | Git, GitHub |
| Deployment | Nginx, Gunicorn |
| Caching | Redis, Memcached |



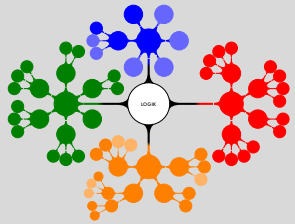


# Python libraries for Web Development (2)

- **Flask-WTF**: Flask-WTF is an extension for handling web forms in Flask applications. It provides utilities for rendering forms, handling form submissions, and performing form validation. Flask-WTF simplifies the process of working with forms and managing user input.
- **PyJWT**: PyJWT is a library for JSON Web Tokens (JWT) authentication. It simplifies the creation, decoding, and verification of JWTs, which are commonly used for authentication and authorization in web applications.
- **Redis-py**: Redis-py is a Python client for Redis, an in-memory data structure store. It allows you to interact with Redis databases and perform operations like storing and retrieving data, caching, and pub/sub messaging.
- **Pydantic**: Pydantic is a data validation and parsing library that simplifies working with complex data structures in Python. It allows you to define data models with type hints and provides automatic data validation, serialization, and deserialization. Pydantic is commonly used in web applications for validating and handling incoming request data.
- **Jinja2**: Jinja2 is a powerful and flexible template engine for Python. It provides a syntax for defining templates with placeholders and logic, which can be rendered dynamically with data. Jinja2 is widely used in web frameworks like Flask and Django to generate HTML pages, emails, and other dynamic content.

These are just a few examples of the many libraries available in the Python ecosystem for web development. Depending on your specific project requirements, you can explore and leverage these libraries to enhance your web development workflow.



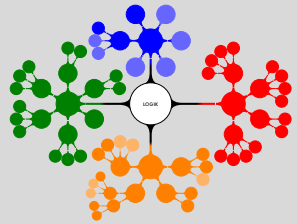


# A Roadmap for Web Development with Python

A roadmap for web development with Python that outlines the key steps and concepts involved in web development with Python:

- **Learn the basics of Python**: Familiarize yourself with the fundamentals of Python programming, including syntax, data types, control structures, and functions. You can refer to online tutorials or books to get started.
- **Understand HTML, CSS, and JavaScript**: Learn the basics of web technologies like HTML for markup, CSS for styling, and JavaScript for client-side interactivity. These are essential for understanding and creating web pages.
- **Choose a web framework**: Select a Python web framework that suits your project requirements. Popular options include Django, Flask, and Pyramid. Each framework has its own strengths and learning curve, so explore their documentation and resources to make an informed decision.
- **Front-end development**: Enhance your web development skills by learning popular front-end libraries and frameworks such as React, Vue.js, or Angular. These frameworks allow you to build interactive user interfaces and communicate with back-end APIs.

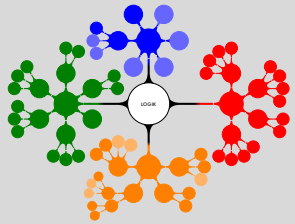




# A Roadmap for Web Development with Python (2)

- **RESTful API development**: If building an API is part of your project, learn about RESTful principles and design patterns. Use your chosen web framework to create APIs that expose data and functionality to other applications or front-end interfaces.
- **Authentication and authorization**: Understand the concepts of user authentication and authorization. Learn how to implement secure user registration, login, and access control mechanisms using your web framework's built-in features or extensions.
- **Testing and debugging**: Gain proficiency in testing your web applications. Learn about unit testing, integration testing, and end-to-end testing. Use tools like pytest, Selenium in Python, or the testing frameworks provided by your chosen web framework to write and execute tests.
- **Deployment and hosting**: Learn how to deploy your web application to a web server or a cloud platform. Understand concepts such as server configuration, deployment automation, security considerations, and scalability. Platforms like Heroku, AWS, or PythonAnywhere are commonly used for web application hosting.





# Version Control with Git

## Why Version Control?

- Track changes in your code
- Collaborate with other developers
- Revert to previous versions if needed

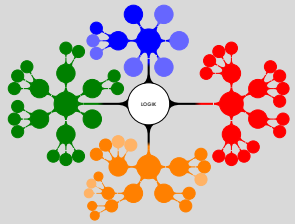
## Git Basics

- **Repository**: Project storage
- **Commit**: Save changes
- **Branch**: Parallel development line
- **Merge**: Combine changes

## Platforms

- GitHub, GitLab, Bitbucket





# APIs and Data Exchange

## What is an API?

An **API (Application Programming Interface)** allows applications to communicate with each other.

## RESTful APIs

- Use HTTP methods (GET, POST, PUT, DELETE)
- Stateless communication
- Resources identified by URLs

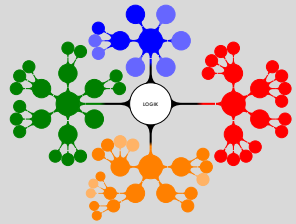
## JSON Format

- Lightweight data format used in web applications

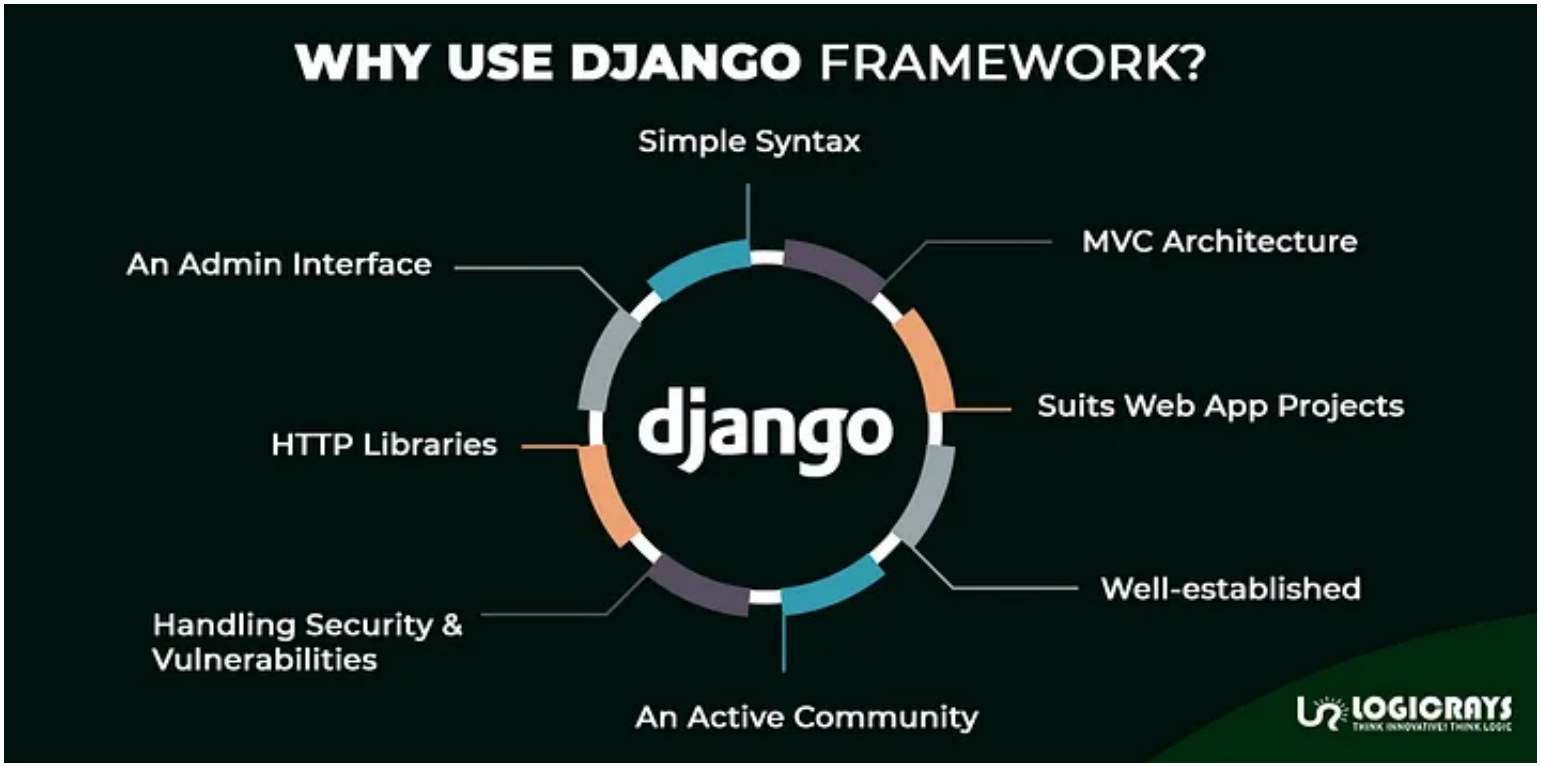
## Example

```
{
  "username": "admin",
  "email": "admin@mail.com"
}
```

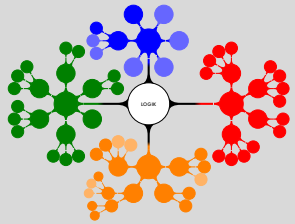




# Django - Advantages

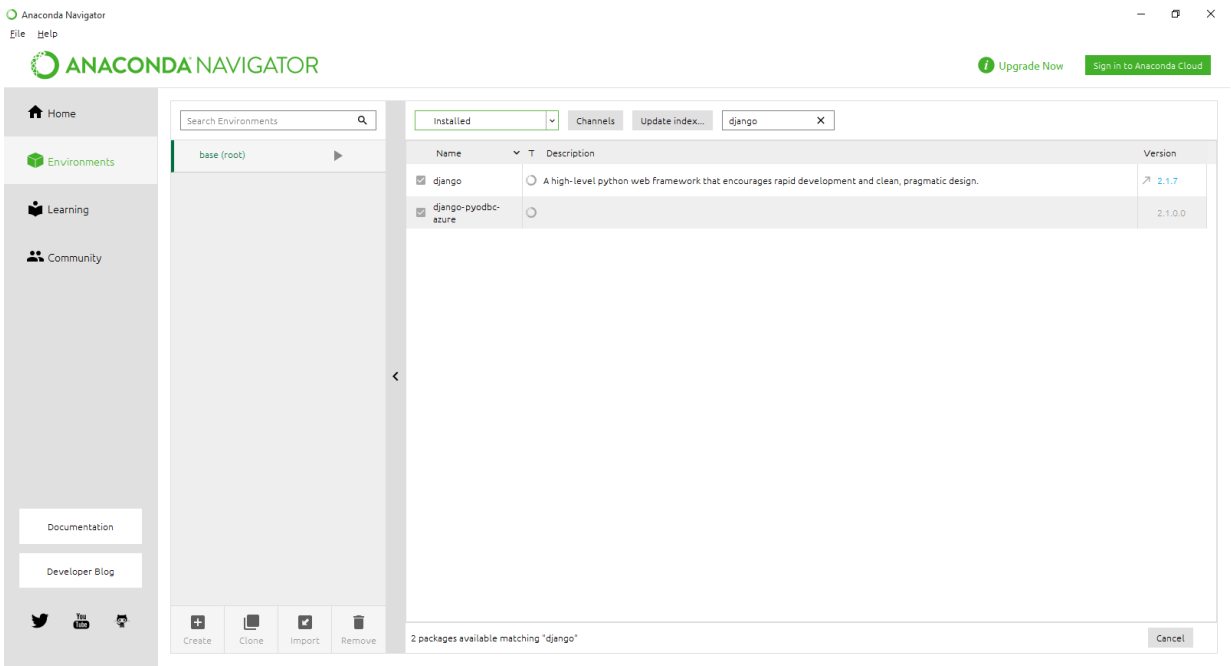




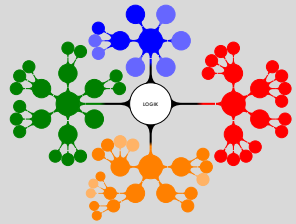



# Install Django with Anaconda

- Open the anaconca navivigator
- Go to the **"Environments"** tab
- Enter **"Django"** in the research text field and select the **"Not installed"** category
- Select and install all Django packages and click apply at the bottom of the page to proceed with the installation.





# Verifying your installation

- Launch the anaconda prompt
- Start a new project

```
django-admin startproject NewWebProj
```

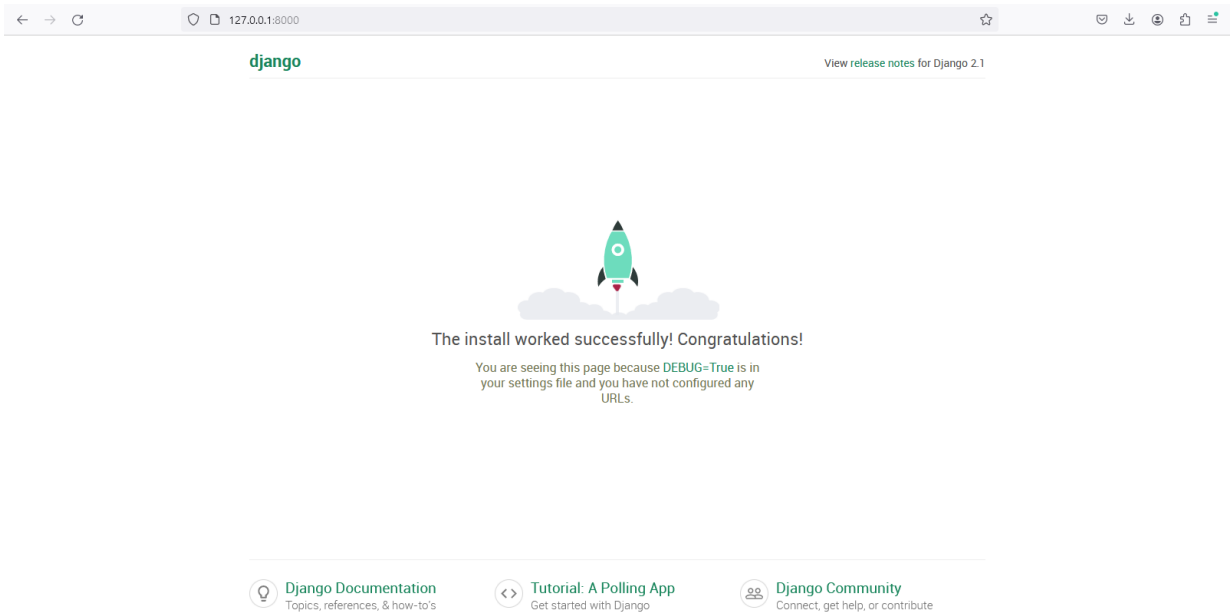
- Go to the project folder

```
cd .\NewWebProj
```

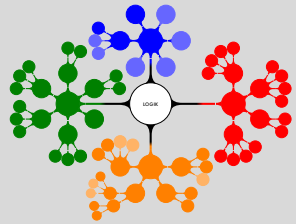
- Start the server

```
python .\manage.py runserver
```

- Launch the app inside the browser using the address: http://127.0.0.1:8000/





# Django Project Structure

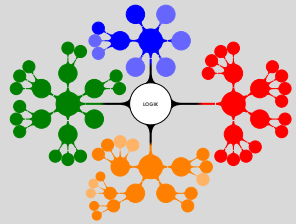
## Main Components

- **models.py**: Defines database structure
- **views.py**: Contains application logic
- **urls.py**: Maps URLs to views
- **templates**/: HTML files for UI

## How It Works

1. User sends request (URL)
2. URL mapped to a view
3. View interacts with model
4. Data sent to template
5. HTML returned to user





# Creating your first app

- Create a new application **myapp** using the following command:

```
python manage.py startapp myapp
```

- Go to the newly created directory

```
cd myapp
```

- Create a new directory called **templates**

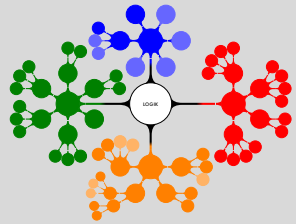
```
mkdir templates
```

- Create the following html file called **index.html** inside templates

C: > Users > Evrad > NewWebProj > myapp > templates > <> index.html > 

```
<!DOCTYPE html>
<html>
<head>
<title>welcome</title>
</head>
<body>
    <center>
        <h1>Welcome to your first Django app</h1>
    </center>
</body>
</html>
```





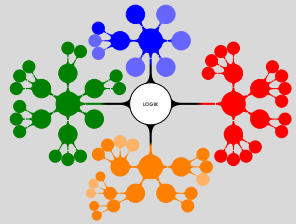
# Creating your first app (2)

- Using the file explorer, go to **NewWebProj/NewWebProj**
- Edit the settings using **settings.py**
- Add the root and templates folders to the project path as shown here below:

C: > Users > Evrad > NewWebProj > NewWebProj > settings.py > ...

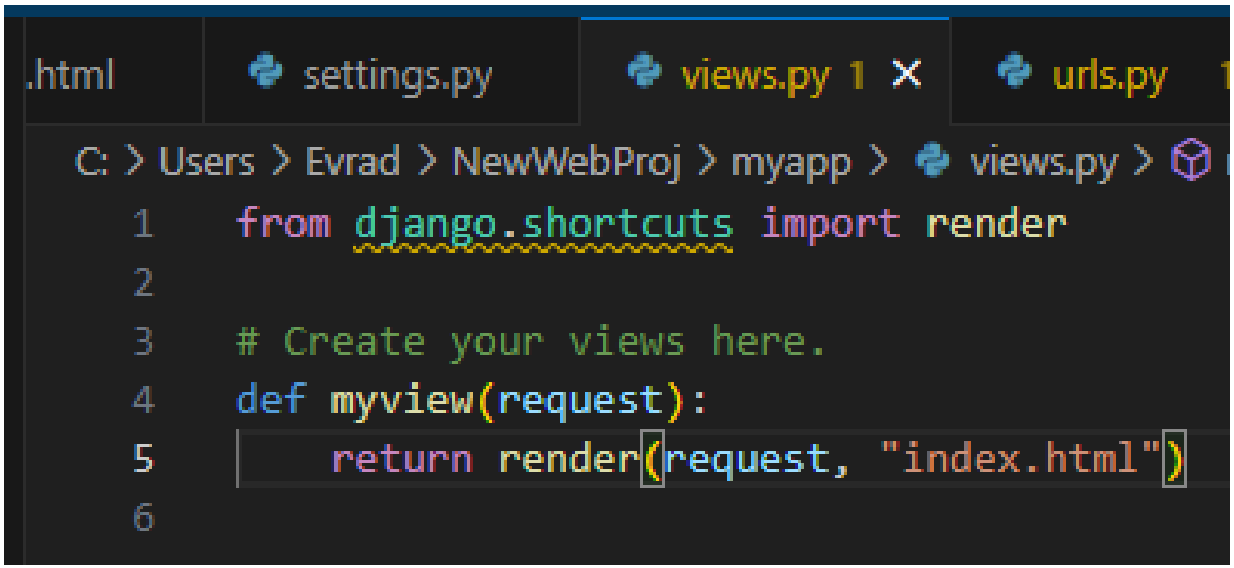
```
41
42  MIDDLEWARE = [
43      'django.middleware.security.SecurityMiddleware',
44      'django.contrib.sessions.middleware.SessionMiddleware',
45      'django.middleware.common.CommonMiddleware',
46      'django.middleware.csrf.CsrfViewMiddleware',
47      'django.contrib.auth.middleware.AuthenticationMiddleware',
48      'django.contrib.messages.middleware.MessageMiddleware',
49      'django.middleware.clickjacking.XFrameOptionsMiddleware',
50  ]
51
52  ROOT_URLCONF = 'NewWebProj.urls'
53
54  TEMPLATES = [
55      {
56          'BACKEND': 'django.template.backends.django.DjangoTemplates',
57          'DIRS': [os.path.join(BASE_DIR, "myapp/templates")],
58          'APP_DIRS': True,
59          'OPTIONS': {
60              'context_processors': [
61                  'django.template.context_processors.debug',
62                  'django.template.context_processors.request',
63                  'django.contrib.auth.context_processors.auth',
64                  'django.contrib.messages.context_processors.messages',
65              ],
66          },
67      },
68  ]
```



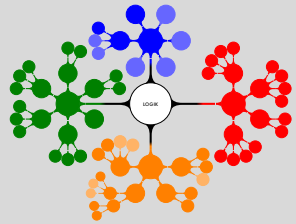


# Creating your first app (3)

- Using the file explorer, go to **NewWebProj/myapp**
- Edit the views using **views.py**
- Add a view for **index.html** as shown here below:





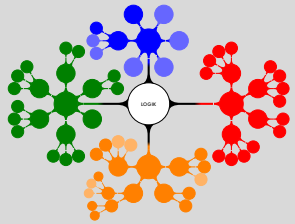
# Creating your first app (4)

- Still inside **NewWebProj/myapp**, edit the routes by creating and editing the file **urls.py**

C: > Users > Evrad > NewWebProj > myapp > urls.py > ..

```
1   from django.urls import path
2   from . import views
3
4   urlpatterns = [
5       path('myview/', views.myview)
6   ]
7
```





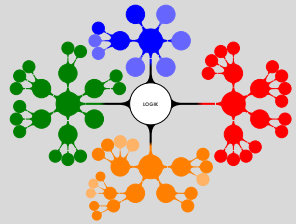
# Creating your first app (5)

- Using the file explorer, go to **NewWebProj/NewWebProj**
- Edit the file **urls.py** as shown here below:

C: > Users > Evrad > NewWebProj > NewWebProj > urls.py > ...

```
15  """
16  from django.contrib import admin
17  from django.urls import path, include
18
19  urlpatterns = [
20      path('myapp/', include('myapp.urls')),
21      path('admin/', admin.site.urls),
22  ]
23
```





# Running your application

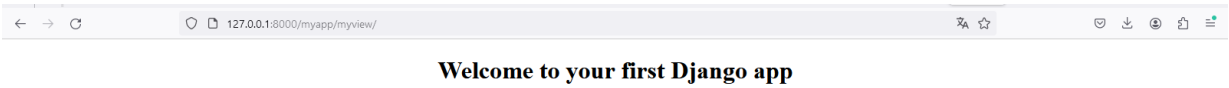
- Using the prompt, go back to the root folder of your project **NewWebProj**

```
cd ..
```

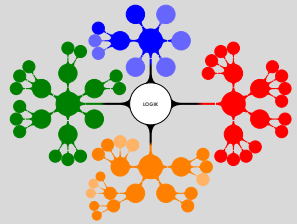
- Run the server **NewWebProj**

```
python manage.py runserver
```

- Enter the following address in the browser to view your web page `http://127.0.0.1:8000/myapp/myview/`





# Flask Web App

To create your first Python web application development example with Flask, Need to follow the below steps:

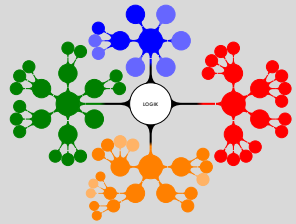
- **Install Flask**: Open your terminal or command prompt and run the following command to install Flask using pip (Python package installer):

```
pip install flask
```

- **Create a Project Folder**: Create a new folder for your project. This will be the root directory for your web application.
- **Create a Python File**: Inside your project folder, create a new Python file. For example, app.py.
- **Import Flask and Create an App Instance**: In app.py, import the Flask module and create an instance of the Flask class. Add the following code in the app.py file:

```
from flask import Flask, render_template, request

app = Flask(__name__)
```



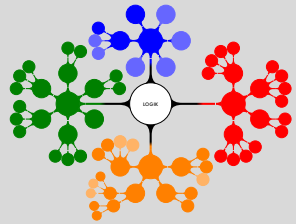


# Flask Web App (2)

- **Define a Route and View Function**: Define a route (URL) and a view function to handle the request. The view function will return the response that will be displayed in the browser. Add the following code:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
if request.method == 'POST':
# Perform login authentication
username = request.form['username']
password = request.form['password']

# Add your authentication logic here
if username == 'admin' and password == 'password':
return 'Login successful!'
else:
return 'Invalid username or password'

# If the request method is GET, render the login template
return render_template('login.html')
```



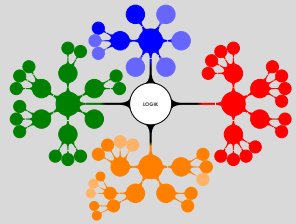


# Flask Web App (3)

- **Create a Login HTML Template**: Inside your project folder, create a new folder named templates. Inside the templates folder, create an HTML file named **login.html**. Add the following code to **login.html**:

```
<!DOCTYPE html>
<html>
<head>
<title>Login</title>
</head>
<body>
<h1>Login</h1>
<form method="POST" action="/login">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required><br><br>
<label for="password">Password:</label>
<input type="password" id="password" name="password" required><br><br>
<input type="submit" value="Login">
</form>
</body>
</html>
```





# Flask Web App (4)

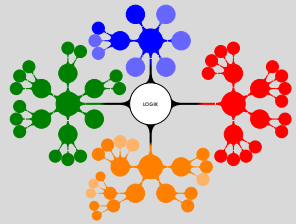
- **Run the Application**: At the end of app.py, add the following code to run the Flask application:

```
if __name__ == '__main__':
app.run()
```

- **Start the Development Server**: Before start server, we need to compile above steps code together, Here is an entire script for app.py file :

```
from flask import Flask, render_template, request

app = Flask(__name__)


@app.route('/login', methods=['GET', 'POST'])
def login():
if request.method == 'POST':
# Perform login authentication
username = request.form['username']
password = request.form['password']

# Add your authentication logic here
if username == 'admin' and password == 'password':
return 'Login successful!'
else:
return 'Invalid username or password'

# If the request method is GET, render the login template
return render_template('login.html')


if __name__ == '__main__':
app.run()
```





# Running the App

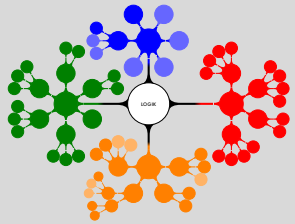
In your terminal or command prompt, navigate to the project folder and run the following command:

```
python app.py
```

- **Open the Login Page**: Open your web browser and enter **http://localhost:5000/log** in the address bar. You should see a login page with a username and password field.
- **Test Login Functionality**: Enter "**admin**" as the username and "**password**" as the password, and click the "**Login**" button. You should see a "**Login successful!**" message. If you enter any other username or password combination, you will see an "**Invalid username or password**" message.

This example demonstrates a basic login page using Flask. You can expand on this by adding database integration, user authentication, session management, and other security measures based on your project requirements.





# From Development to Production

## Environments

- **Development**: Local testing
- **Production**: Live application for users

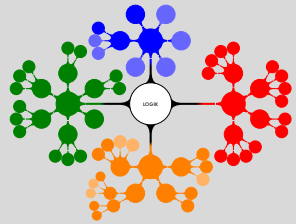
## Python Web Servers

- **WSGI**: Traditional Python web interface (Django, Flask)
- **ASGI**: Async support (FastAPI, Django async)

## Deployment Stack

- Application server: Gunicorn
- Web server: Nginx
- Environment variables (.env)





# Web Security Standards: OWASP

## What is OWASP?

**OWASP (Open Web Application Security Project)** provides best practices for securing web applications.

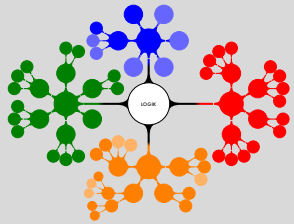
## Common Risks (OWASP Top)

- Injection (SQL, command injection)
- Broken Authentication
- Sensitive Data Exposure
- Security Misconfiguration
- Cross-Site Scripting (XSS)

## Why It Matters

- Industry standard for secure development
- Helps prevent major vulnerabilities



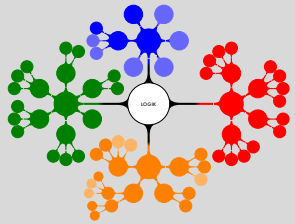


# Python Web Apps and Security

Web applications are vulnerable to various security threats. Some essential security practices include:

- **Input Validation**: Validate and sanitize all user inputs to prevent common attacks like SQL injection and cross-site scripting (XSS).
- **Password Security**: Store passwords securely using techniques like hashing and salting to protect user credentials.
- **Cross-Site Request Forgery (CSRF) Protection**: Implement measures like CSRF tokens to prevent unauthorized requests.
- **Secure Session Management**: Use secure session management techniques to protect user sessions, such as using session cookies with secure flags and implementing session timeouts.

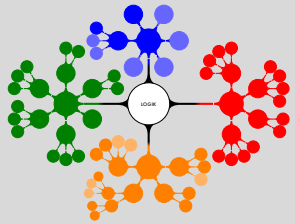




# Python Web Apps and Security (2)

- **Use HTTPS**: Ensure all data transmitted between your server and clients is encrypted using SSL/TLS. This protects your users' data from eavesdropping. You can use tools like Let's Encrypt to get a free SSL certificate.
- **Regular Audits**: Conduct regular security audits and penetration testing to identify and fix potential vulnerabilities.
- **Dependencies**: Regularly update your application's dependencies using tools like 'pip'. Vulnerabilities in outdated packages can be exploited, so stay informed and use tools like 'pyup' or 'dependabot' to get notified about outdated packages.
- **Content Security Policy (CSP)**: Implement CSP headers to reduce the risk of XSS attacks by controlling which resources can be loaded.

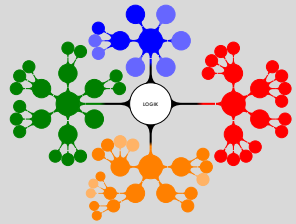




# Specific vulnerabilities

- **httpoxy** is a set of vulnerabilities that can affect Python web application servers via HTTP requests.
- **Heartbleed** is a vulnerability in OpenSSL implementations that must be patched for any systems you run otherwise you are at serious risk for data leakage.
- **Meltdown** and **Spectre** are x86 architecture problems caused by exploiting CPU branch-prediction implementations.
- **SQL injection** remains a persistent threat to web applications, including those built with Python. Attackers exploit inadequately sanitized user inputs to manipulate SQL queries, potentially gaining unauthorized access to databases and sensitive information.





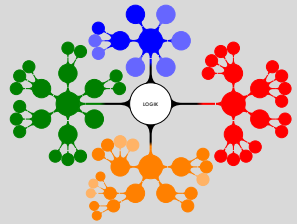
# Specific vulnerabilities (2)

- **Cross-Site Scripting (XSS)** allow attackers to inject malicious scripts into web pages viewed by other users. This can lead to the theft of sensitive user data or the manipulation of user interactions.

- **Cross-Site Request Forgery (CSRF)**: CSRF attacks trick authenticated users into unknowingly performing actions on a web application without their consent. Attackers exploit the trust established between a user and a site.

| Vulnerability | Description | Example |
|---|---|---|
| **SQL Injection** | Manipulation of SQL queries through user input | Username: ’ OR ’1’=’1 |
| **XSS** | Injection of malicious scripts into web pages | Payload: <script> maliciousCode() </script> |
| **CSRF** | Forging unauthorized requests using user trust | Exploit: <img src="..."> |



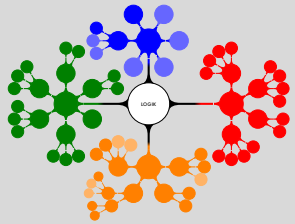


# Secure Coding Practices

Adhering to secure coding practices mitigates a plethora of potential vulnerabilities:

- Parameterized queries, facilitated by libraries like 'sqlite3', defend against SQL injection by separating data from SQL commands.
- Output encoding, accomplished through functions like 'html.escape()', prevents Cross-Site Scripting (XSS) attacks by rendering user inputs as harmless text.
- Equally, avoiding functions like 'eval()' and 'exec()' prevents unintended execution of arbitrary code, reducing the risk of code injection attacks

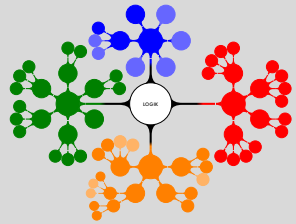




# Security tools

1. **lynis** is a security audit tool that can run as a shell script on a Linux system to find out its vulnerabilities so that you can fix them instead of allowing them to be exploited by malicious actors.
2. **Charles** is an HTTP proxy for inspecting headers, requests and responses for all traffic that flows through it.
3. **TLS Observatory** provides a suite of security tools for analyzing and inspecting Transport Layer Security (TLS) services. There is also a hosted version you can use at observatory.mozilla.org.
4. **WIG** contains tools for gathering wireless data via Wifi protocols.
5. **HTTP Evader** is an automated testing tool for checking firewalls to ensure they are protecting the appropriate ports and payloads.
6. **Security monkey** monitors for changes to AWS, Google Cloud, GitHub and other infrastructure systems.





# Best Practices for Python Web Development

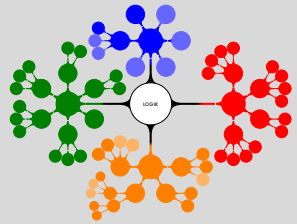
When it comes to Python web development, there are several best practices you can follow to ensure clean, efficient, and maintainable code. Here are six essential points to consider:

- **Use a Web Framework**: Python offers a wide range of web frameworks like Django, Flask, and Pyramid. These frameworks provide essential tools and features for web development, such as routing, request handling, and template engines. Choosing a framework helps you structure your codebase and promotes code reuse.

- **Follow the MVC Pattern**: Model-View-Controller (MVC) is a software architectural pattern commonly used in web development. It helps separate the concerns of your application by dividing it into three components: models (representing data and business logic), views (handling user interface), and controllers (managing the flow between models and views). Adhering to this pattern makes your code more modular and maintainable.

- **Use Virtual Environments**: Virtual environments create isolated Python environments for your projects, allowing you to manage dependencies and avoid conflicts between packages. Tools like virtualenv or Python's built-in venv module enable you to create and activate virtual environments. This practice ensures project-specific dependencies and avoids cluttering your global Python installation.





# Best Practices for Python Web Development (2)

When it comes to Python web development, there are several best practices you can follow to ensure clean, efficient, and maintainable code. Here are six essential points to consider:

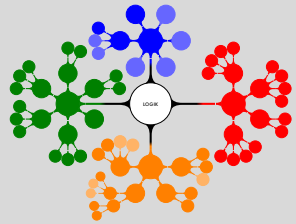
- **Employ Database Abstraction Layers**: When working with databases, use an ORM (Object-Relational Mapping) library like SQLAlchemy. ORM libraries abstract the database layer, allowing you to interact with the database using Python objects and queries. This approach simplifies database operations, improves security by preventing SQL injection, and facilitates switching between different database systems.

- **Write Unit Tests**: Unit testing is crucial for maintaining code quality and catching potential bugs early. Python has a built-in testing framework called unittest, along with third-party libraries like pytest. Write comprehensive unit tests for your web application's components, including models, views, and controllers. Automating tests helps ensure that future code changes don't introduce unexpected issues.

- **Handle Security**

- **Regularly Update Dependencies**: Keep your Python packages and web frameworks up to date to ensure you're using the latest security patches.





# Performance and Scalability

## Improving Performance

- **Caching**: Store frequently used data (Redis, Memcached)
- **Database Optimization**: Indexing, query optimization

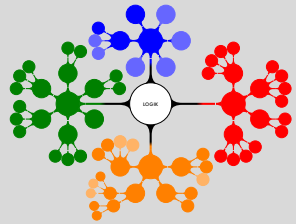
## Scalability Techniques

- **Load Balancing**: Distribute traffic across servers
- **Horizontal Scaling**: Add more servers
- **Asynchronous Processing**: Handle multiple requests efficiently

## Tools

- Redis, Celery, Nginx



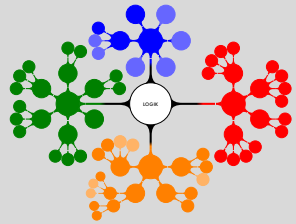


# Benefits of Secure Web Development

Implementing secure coding practices in Python web applications is not just a precaution; it's a necessity. The benefits of secure web development are immense:

- **User Trust**: Secure applications build user confidence. When users know their data is safe, they are more likely to engage with the web application.
- **Data Protection**: Robust security measures ensure that sensitive user data is protected from breaches, thus maintaining data integrity and confidentiality.
- **Business Reputation**: Security breaches can tarnish a company's reputation. Secure practices safeguard the business's image and credibility.
- **Compliance with Regulations**: Adhering to security standards and practices ensures compliance with legal and regulatory requirements, avoiding potential fines and legal issues.
- **Prevention of Financial Losses**: Security breaches can be costly. Secure development practices help avoid costs associated with data breaches, such as legal fees, compensations, and loss of revenue.





# Hands-on Lab: Build a Simple Web App

## Objective

Apply the concepts learned to build a simple Python web application.

## Tasks

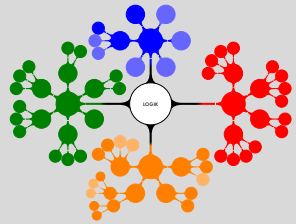
1. Create a web application using Flask or Django
2. Define at least one route (URL)
3. Display dynamic content using a template
4. Connect the application to a database (SQLite)
5. Implement a simple form (e.g., login or contact form)

## Expected Outcome

A functional web application demonstrating:

- Routing
- Template rendering
- Basic data handling





# Summary

1. Web development involves creating websites and web applications, and Python is a powerful language that is well-suited for this purpose.
2. Python's simplicity, readability, and extensive ecosystem make it an excellent choice for web development projects.
3. Python provides a solid foundation for web development, enabling developers to build feature-rich and scalable web applications efficiently.